

```

1  # Note 08: KNN Algorithm
2  # =====
3
4  # Import Libraries
5  import pandas as pd
6  from sklearn.model_selection import train_test_split
7  from sklearn.neighbors import KNeighborsClassifier
8  from sklearn.metrics import accuracy_score, confusion_matrix, mean_absolute_error,
mean_squared_error, r2_score
9  import matplotlib.pyplot as plt
10 import numpy as np
11 from sklearn.preprocessing import StandardScaler, MinMaxScaler
12 from sklearn.neighbors import KNeighborsRegressor
13
14 # 01. KNN Algorithm - Classification Problem
15 # =====
16
17 # Load Dataset
18 df = pd.read_csv("knn_dataset.csv")
19
20 # Define X and Y Variables
21 x = df.drop("Class", axis=1) # axis = 1 [Column]
22 y = df["Class"]
23
24 # Split Dataset
25 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2,
random_state = 42)
26
27 # Standardization
28 # =====
29
30 # scaler = StandardScaler()
31 # x_train = scaler.fit_transform(x_train)
32 # x_test = scaler.transform(x_test)
33
34 # Normalization / Min - Max Scaling
35 # =====
36
37 # scaler = MinMaxScaler()
38 # x_train = scaler.fit_transform(x_train)
39 # x_test = scaler.transform(x_test)
40
41 # Find Best K Value [K Values vs Accuracy Score Graph]
42 k_values = range(1, 11)
43 accuracies = []
44
45 # Loop to Find Best K Value
46 for k in k_values:
47     knn = KNeighborsClassifier(n_neighbors=k)
48     knn.fit(x_train, y_train)
49     y_pred = knn.predict(x_test)
50     accuracies.append(accuracy_score(y_test, y_pred))
51     # acc = (np.sum(y_pred == y_test) / len(y_test)) * 100
52     # accuracies.append(acc) # Manual Method
53
54 # Plot and Pick the k with the highest accuracy
55 plt.figure()
56 plt.scatter(k_values, accuracies, marker = "o", label = "Accuracy Scores Data Points",
color = "red")
57 plt.plot(k_values, accuracies, color = "blue", label = "Accuracy Scores Line")
58 plt.xlabel("K Values")
59 plt.ylabel("Accuracy Scores")
60 plt.title("K Values vs Accuracy Scores")
61 plt.grid(True, alpha = 0.3)
62 plt.legend()
63 plt.show()
64
65 # Print Best K Value
66 best_k = list(k_values)[np.argmax(accuracies)]
67 best_acc = max(accuracies)
68 print("\nBest K Value      : ", best_k)
69 print("Best Accuracy Score : ", best_acc)

```

```

70
71 # Create KNN Model
72 knn = KNeighborsClassifier(n_neighbors = 2, weights = "uniform", metric = "minkowski",
73     algorithm = "auto", leaf_size = 30, p=2, n_jobs = None, metric_params=None)
74
75 # Fit the Model (Train the Model)
76 knn.fit(x_train, y_train)
77
78 # Predict
79 y_pred = knn.predict(x_test)
80
81 # Evaluation
82 print("\n=====")
83 print("Evaluation Report - KNN Classifier")
84 print("=====\n")
85 print("Accuracy Score as a Percentage:", round((accuracy_score(y_test, y_pred) * 100),
86     2), "\b%")
87
88 # Confusion Matrix
89 print("\n=====")
90 print("Confusion Matrix")
91 print("=====\n")
92 print(confusion_matrix(y_test, y_pred), "\n")
93
94 # Predict a New One
95 new_one = pd.DataFrame([[6.1, 5.9, 5.6]], columns = ["Feature1", "Feature2",
96     "Feature3"])
97 prediction = knn.predict(new_one)
98
99 # Print Prediction Result
100 print("Expected Class : Class 1")
101 print("Predicted Class : Class", prediction[0])
102
103 # 02. KNN Algorithm - Regression Problem
104 # =====
105
106 # Create sample data
107 X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9], [10] ]) # 10 x 1 2D Matrix
108 y = np.array([10, 15, 20, 24, 30, 35, 40, 45, 50, 55])
109
110 # Train - Test Split
111 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
112     42)
113
114 # Feature scaling
115 scaler = StandardScaler()
116 X_train = scaler.fit_transform(X_train)
117 X_test = scaler.transform(X_test)
118
119 # Create KNN Regressor
120 knn = KNeighborsRegressor(n_neighbors=3, weights='uniform')
121
122 # Train
123 knn.fit(X_train, y_train)
124
125 # Predict
126 y_pred = knn.predict(X_test)
127
128 # Evaluation
129 mae = mean_absolute_error(y_test, y_pred)
130 mse = mean_squared_error(y_test, y_pred)
131 rmse = np.sqrt(mse)
132 r2 = r2_score(y_test, y_pred)
133
134 # Print Results
135 print("\n=====")
136 print("Evaluation Report - KNN Regressor")
137 print("=====\n")
138
139 print("MAE :", mae)
140 print("MSE :", mse)

```

```
138 print("RMSE:", rmse)
139 print("R2 :", r2)
```